

DevOps Interview — Detailed Questions & Model Answers

Prepared from your interview transcript and expanded into technically stronger answers for future interviews.

Focus: The interviewer valued your issue-analysis ability and asked you to improve precise clarification when a question is outside your immediate knowledge.

Linux / Shell

How would you rate yourself in Linux out of 5? Why?

I would rate myself around 3.5–4/5. I am comfortable with day-to-day production troubleshooting: processes, CPU/memory/disk, permissions, networking basics, logs, shell commands and service management. I am still strengthening advanced Linux internals and performance tuning.

Interview note: Be honest, then support the rating with concrete skills.

What is the difference between `./script.sh` and `sh script.sh`?

`./script.sh` executes the file directly, so it needs execute permission and normally a valid shebang. `sh script.sh` explicitly invokes the `sh` interpreter, so the file does not need execute permission. The command determines the interpreter.

Interview note: Remember: `chmod` changes permissions; `sh/bash` interprets a readable script.

How can you run a script without execute permission?

Use an interpreter explicitly: `sh script.sh` or `bash script.sh`. Direct execution with `./script.sh` requires the execute bit.

Scheduling / Automation

Have you used scheduling tools for scheduling jobs?

Yes. I have used scheduled automation in Jenkins for operational/cleanup activities. Linux `cron` or `systemd` timers can also be used for host-level scheduled tasks.

What kind of scheduling tools have you used?

For CI/CD, Jenkins scheduled jobs using `cron` expressions. For OS-level maintenance, Linux `cron` is a standard option. The choice depends on where the automation belongs.

Oracle / Database Troubleshooting

A stuck database session is blocking a chain of executions. What solution would you provide to the DBA?

First identify the blocking session and the sessions it blocks. Provide `SID`, `SERIAL#`, username, SQL ID, status, blocking-session information, transaction age and relevant SQL. Confirm whether it is a legitimate long transaction. If it is confirmed as the blocker and termination is approved, the DBA can use `ALTER SYSTEM KILL SESSION 'sid,serial#' IMMEDIATE`. Then monitor rollback, confirm blocked sessions are released and verify application recovery.

Interview note: Flow: identify blocker → understand transaction → confirm impact → DBA approval/kill → monitor rollback → verify → document.

Have you faced stuck-session/database-lock issues before?

A safe answer is: I have encountered database-side blocking symptoms from an application perspective. I identify the affected operation, collect blocking/blocked-session details, check for long-running transactions or locks and coordinate with the DBA. If it repeats, I track the pattern and work with the DBA/application team on a permanent resolution.

Interview note: Do not claim DBA-level hands-on work unless you actually performed it.

Terraform

Do you have hands-on experience with Terraform?

Yes. I have used Terraform to provision/manage AWS infrastructure, organize reusable modules, maintain state and variables, review changes with terraform plan and apply controlled changes. I also understand importing existing resources into Terraform management.

If infrastructure is static and already created, why do we need Terraform?

Terraform provides Infrastructure as Code: version control, reviewable changes, repeatability, drift detection, standardization and reproducibility. Stable infrastructure still benefits because future changes can be controlled and audited instead of being made manually.

If infrastructure was created manually and is already stable, how do you start using Terraform?

I would not recreate it. I would inventory the existing resources, write Terraform configuration representing the desired architecture, import the existing resources into state, run terraform plan and reconcile differences until there are no unintended changes. Then future changes can be managed through Terraform and CI/CD.

Interview note: Flow: resource → code → import → plan → reconcile → manage.

If everything is already stable, how is Terraform useful?

It becomes the controlled source of truth for infrastructure. Future changes to networking, security groups, instance types, IAM, scaling or other resources can be reviewed in Git, planned and applied consistently. It also helps detect drift and reproduce environments.

If Terraform created a resource and you run terraform apply again, will it recreate it?

Normally no. Terraform uses state to map configuration to real resources, reads current attributes through the provider and calculates a plan. If there are no changes, the plan is no changes. If something changed, Terraform determines whether it can update in place or must replace the resource. Some attributes require replacement.

Interview note: Terraform does not rely only on the state file; provider-read data and resource lifecycle rules matter.

You created an instance with 4 GB RAM and 64 GB storage. Can you change it to 6 GB RAM and 128 GB storage with the same name?

Yes, subject to the resource capabilities. In AWS, CPU/RAM are normally changed through the EC2 instance type and EBS capacity through volume size. Some changes are in-place and some can require replacement. Always inspect terraform plan before apply.

How does Terraform work in the background when updating an existing resource?

Terraform loads configuration, identifies the resource in state, reads current remote attributes through the provider, compares current versus desired configuration and builds a plan. During apply, the provider makes the required cloud API calls. The change may be in-place or may require replacement depending on the resource attribute.

What Terraform file/folder structure do you use?

A clean structure can include main.tf, variables.tf, outputs.tf, providers.tf and a modules/ directory containing reusable VPC, EC2, IAM and other modules. Environment-specific values can be supplied through tfvars or environment conventions.

Kubernetes / EKS

Explain your Kubernetes experience.

I have worked with Kubernetes on AWS EKS and deployments using Helm and GitOps. My responsibilities include deployments, Services, Ingress, configuration, storage, troubleshooting, resource management and monitoring. I have also worked with Argo CD and Prometheus/Grafana.

Explain how your application architecture works within Kubernetes.

Our microservices platform runs on EKS. External traffic enters through the ingress/load-balancing layer and is routed to Kubernetes Services and Pods. Pods communicate with other services and backend dependencies such as RDS and Redis. Images are stored in ECR. Helm packages configuration, Argo CD reconciles Git desired state and Prometheus/Grafana provide monitoring.

Interview note: Mention the 50+ microservices when relevant.

Argo CD / GitOps

Does Argo CD use hooks or polling to pull changes from Git?

Argo CD monitors Git and can detect changes through repository polling and webhook notifications. Git contains the desired state. Argo CD compares desired state with live Kubernetes state and determines whether the application is Synced or OutOfSync.

How does Argo CD rewrite changes in the manifest file?

Argo CD normally does not rewrite the Git manifest itself. In a Jenkins + Argo CD workflow, Jenkins may update the image tag in the Git deployment/Helm values repository and commit it. Argo CD detects the Git change, renders the manifests/Helm chart, compares desired state with live state and applies the required Kubernetes changes.

Interview note: Key sentence: Jenkins changes Git; Argo CD reconciles Git to the cluster.

Docker Images / Layers

During production deployment, an image/ConfigMap change reports a component conflict. How would you troubleshoot it?

First identify where the error occurs: Docker build, registry push, image pull, Kubernetes admission, container startup or application runtime. Inspect Jenkins logs, Dockerfile, image digest, Kubernetes events and container logs. Render the exact Helm/manifests and verify the intended image tag/digest and ConfigMap. Compare working and failing versions. If the image is suspect, rebuild cleanly with a new immutable tag, push to ECR, verify the digest and redeploy.

If it is a single image with a component conflict, what would you check?

First clarify what 'conflict' means because build/cache issues, registry layer errors, filesystem overlay behavior and application-level conflicts are different. Then inspect the exact error, image manifest/digest, Dockerfile, cache, registry status and container logs.

Interview note: Good clarification: Do you mean a build/cache issue, registry layer issue, or runtime filesystem conflict?

What happens to layers when an image is rebuilt?

Docker image layers are immutable. Unchanged build steps can reuse cached layers. If an instruction or its inputs change, that step and dependent later steps may rebuild. The old image is not modified; the new image references a new combination of layers.

When code changes and docker build runs again, what happens?

Docker evaluates the Dockerfile and reuses cache where instructions and inputs are unchanged. After a cache miss, that step and later dependent steps may rebuild. A common optimization is to install dependencies before copying frequently changing source code.

Does Docker remove/modify layers from an existing image when building a new image?

No. Existing image layers are immutable. A new image can reuse old layers and add new ones. Local garbage collection may later remove unreferenced layers, but that is different from modifying an existing image.

What can cause a 'layer conflict' type of problem?

The exact cause depends on the error. Possible areas include registry upload/download failures, incomplete/corrupted layers, manifest/digest mismatch, cache/build problems, storage problems or an application/runtime problem described as a layer conflict. Two layers containing the same path do not automatically cause Docker to crash.

If two layers change the same filesystem path, what happens?

Multiple layers can affect the same path. Overlay filesystem semantics allow upper layers to add or replace lower-layer content. This is normal and is not automatically an error. If someone reports a 'layer conflict', ask for the exact error before assuming overlapping paths are the cause.

Is Docker's layering concept idempotent?

A precise answer is that image layers are immutable and content-addressed, and identical layers can be reused. But 'layers are idempotent' is not a complete technical statement. Idempotency describes repeated operations producing the

same resulting state; reproducible builds require controlled inputs and build processes.

Interview note: Do not memorize the transcript's claim that overlapping paths inherently create a Docker layer conflict; that is not generally correct.

Kubernetes CrashLoopBackOff

What are your troubleshooting steps for CrashLoopBackOff?

1) kubectl get pod for status/restarts. 2) kubectl describe pod for events, probes and mounts. 3) kubectl logs for current logs. 4) kubectl logs --previous for the previous crashed container. 5) Check exit code/reason. 6) Validate ConfigMaps/Secrets. 7) Verify image/tag/pull status. 8) Check liveness/readiness/startup probes. 9) Check CPU/memory and OOMKilled. 10) Check PVC/volume mounts and dependencies. Then fix, redeploy and monitor the rollout.

What repeated issues have you seen causing CrashLoopBackOff?

Common causes include application startup failures, incorrect configuration, missing environment variables or Secrets, failed PVC/volume mounts, OOMKilled containers, incorrect liveness/startup probes, dependency failures and application exceptions. CrashLoopBackOff is a symptom of repeated container failure; logs, events and container state identify the root cause.

Final Interview Strategy

Direct answer first: Answer the question in one or two sentences before expanding.

Troubleshooting structure: symptom → evidence → isolate component → validate root cause → least-risk fix → verify → prevent recurrence.

Clarify precisely: If a term is ambiguous, identify the exact dimension you do not understand instead of asking the interviewer to repeat the whole question.

Do not bluff: For DBA-owned work, explain what you can analyze and what you would provide to the DBA.

Priority for this role: Linux, Oracle session/lock troubleshooting, Docker internals, Terraform lifecycle/state/provider behavior, Kubernetes troubleshooting and production incident management.

Production mindset: I first identify the exact failure point, collect evidence, isolate the component, validate the root cause, apply the least risky fix, verify recovery and document/prevent recurrence.